Capital Circuit - Monopoly Game

CSC301 Group 4 | console Monopoly in Java (Abu Dhabi edition)

Data structures used in the Monopoly Game and its explanation

Course	CSC301 Data Structures and Algorithms - Dr Zakaria Khiati, Abu Dhabi University
Group / member	Group 4 | CS (Caron Sylvia)
Code base	25 Java files in C:\Claude\Monopoly\code (default package), board and card files in code\data; branch CS-Monopoly of CSC301-Group4-Monopoly
Scope	Core game (brief Section 3) with console board view, colours and standings. Twists deferred until the group's assignment is confirmed; bonus B4 and language toggle planned
Verification	53 automated tests (incl. every Section 8.2 scenario), 1,000 seeded games, automatic constraint check: PASS
Brief reference	Section 4 (required data structures), 4.1 (minimum operations), 4.2 (constraints), 5 and 5.1 (classes, recursive methods), 8.1 (what every member must understand)

1. Summary

1.1 Overview

The game is a console version of simplified Monopoly on the group's own 24-square Abu Dhabi board. It loads the board and cards from text files, lets 2 to 6 players take turns with two dice, and applies every rule in Section 3 of the brief: buying, rent (doubled for a full colour group), transit rent, tax, Chance and Community cards, Jail, doubles, bankruptcy, the end of the game and the final ranking. Each event is written to the Appendix C game log. A colour board view shows the squares, owners, tokens, each player's cash and properties, the turn queue and the last turn's events.

The brief fixes which data structure does which job, and the code follows it exactly. The **board** is the group's own **circular doubly linked list**: one node per square, the last node linked back to GO, and every player's position held as a node reference rather than an index. **Turn order** and the two **card decks** use the group's own **linked queue** on singly linked nodes. **ArrayList** holds data that needs indexed access and a changing size: each player's squares, the list of players for the ranking and the card list before shuffling. **Recursion**, with no loops, implements the eight required methods for movement, doubles, landing, rent, net worth and board searches.

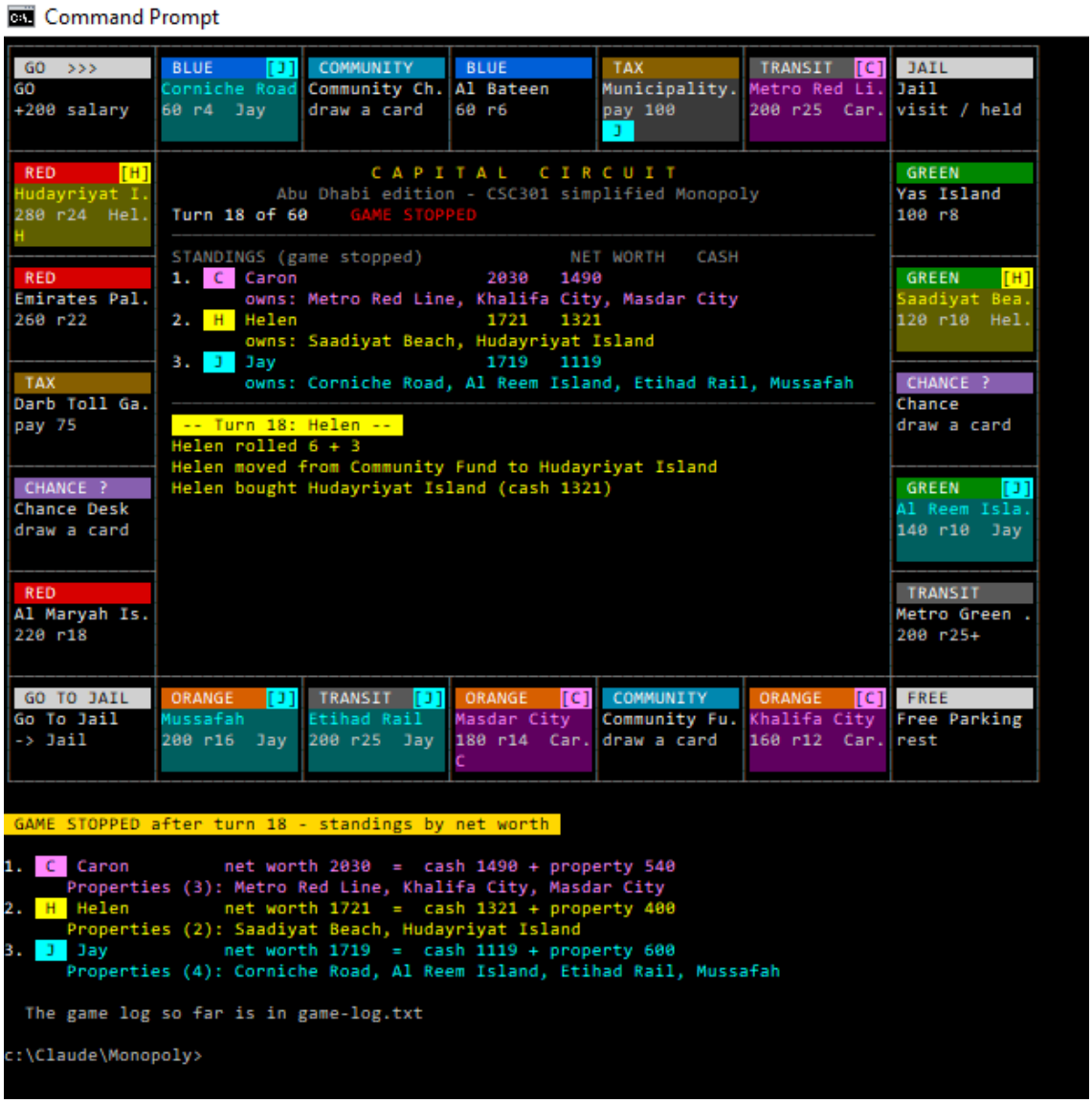
2	3	24	8	10	0
own generic structures	LinkedQueue instances	board nodes in the ring	required recursive methods	further recursive helpers	banned collections used

1.2 Data structures at a glance

Data structure	Implementation in the code	Game functions it serves	Key operations and cost
Circular doubly linked list	Own generic `CircularDoublyLinkedList<T>` with `Node<T>` (data, next, prev); wrapped by `Board`	The board; player positions; dice and card movement forward and backward; salary at GO; colour-group check; MOVE_TO / NEAREST searches; drawing the board	addLast, insertAfter, remove, getHead, size: O(1). Moving k squares: O(k). Searches: O(n)
Queue (singly linked)	Own generic `LinkedQueue<T>` with private `QNode<T>` (data, next): `Game.turnQueue`	Turn order; removing a bankrupt player; the order of PAY_EACH and COLLECT_EACH; the game-over check	enqueue, dequeue, peek, isEmpty, size: O(1). remove(p) and copyInto: one rotation, O(n)
Queue (same class)	`Game.chance`, `Game.community`	Chance and Community decks: draw from the front, return to the back; JAIL_FREE cards returned to their deck	Draw and return: O(1)
ArrayList	`java.util.ArrayList`	Each player's owned squares and held JAIL_FREE cards; all players (join order, ranking); card list before shuffling; bankruptcy order; log lines; scripted dice	get: O(1); add and remove-last: O(1) amortised; sort: O(n log n)
Recursion	8 required methods + 10 helpers, no for/while/do	Movement, doubles, landing and card chains, rent, net worth, board searches, queue rotation, bankruptcy clean-up	See Section 2.4

1.3 The game on screen

Screenshot taken by CS from the running game in the Windows Command Prompt: turn 18 of a three-player game (Caron, Helen, Jay), just after Q was pressed. The centre shows the standings; the report under the board lists each player's net worth and properties.



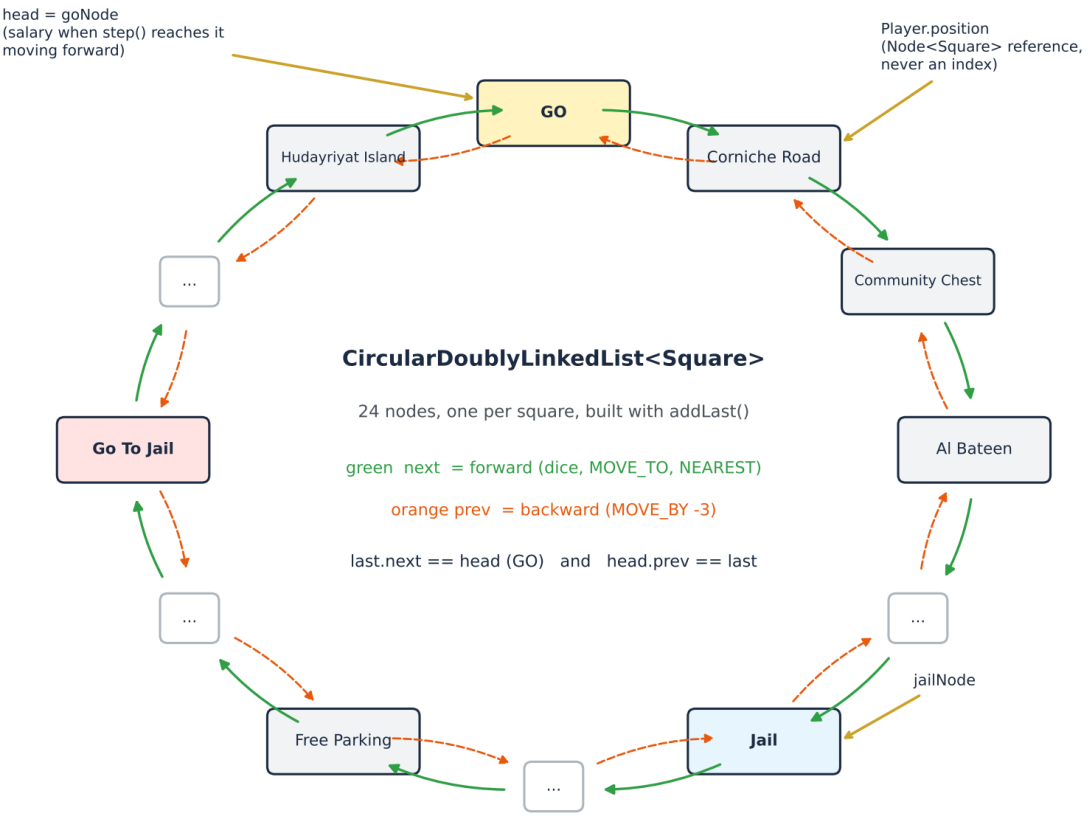
What you see on screen	Data structure behind it
The ring of 24 squares, drawn clockwise from GO in the top-left corner	**Circular doubly linked list**: BoardView walks `next` from the head (GO) once round and places each square on the screen
A player's token (C, H, J) on a square	`Player.position`, a `Node<Square>` reference into the ring
Owner marker [C] / [H] / [J], square tinted in the owner's colour	`OwnableSquare.owner` on the node's square, plus the owner's **ArrayList** of squares
"owns: ..." lines and the property list in the report	Board walked in ring order and filtered by owner; count from `Player.owned` (**ArrayList**)
Ranking by net worth, with cash and property value	**ArrayList** of players sorted with `Collections.sort` (Player implements Comparable); property value by recursive `netWorth(owned, 0)`
Turn queue line and "Next to play" (during play)	**LinkedQueue<Player>**: `peek()` and a one-rotation `copyInto()`
Events of the last turn ("Helen rolled 6 + 3 ...")	Appendix C log lines kept in an **ArrayList<String>** by GameLogger and turned into sentences

2. Detailed explanation

2.1 Circular doubly linked list - the board

What it is. Each Node<Square> holds one square and two links: `next` (the following square) and `prev` (the one before). The last square's `next` is GO and GO's `prev` is the last square, so the ring has no ends: a player keeps going round without any index arithmetic or `% size`, which the brief forbids for movement (4.2).

Why this structure. Every move in Monopoly is "go to the next square" or "go back one square", which is a single pointer hop in either direction. Passing GO is detected by node identity (the next node is goNode). Positions are node references, so if squares are inserted or removed during play (twists T3 Expansion, T2 Earthquake) no player's position has to be renumbered. An ArrayList was rejected because movement would need index arithmetic and inserting a square would shift every later index (O(n)); a singly linked list was rejected because backward moves (MOVE_BY -3, and Mirage in T5) would need a full lap.



Operations (brief 4.1)

Operation	What it does	Used for in the game	Cost
addLast(T)	Links a new node between the last node and GO	FileLoader builds the board in file order, GO first	O(1)
insertAfter(Node, T)	Links a new node directly after a given node	Ready for T3 Expansion (squares inserted after turn 20)	O(1)
remove(Node)	Unlinks a node; if it is the head, the head moves on	Ready for T2 Earthquake (a property square destroyed)	O(1)
getHead()	Returns the GO node	Start of every search and of the board drawing	O(1)
size()	Number of squares	Bound for searches (remaining), screen layout	O(1)

Game functions built on the ring

- **Board set-up** - `FileLoader.loadBoard` reads each line, creates the right Square subclass and calls `addLast`; `Board` keeps `goNode` (the head) and finds `jailNode` with the recursive `findNearest`.
- **Dice movement and salary** - `Game.movePlayer` calls the recursive `Board.step(current, total, p)`, which follows `next` one square per call and pays 200 when the next node is GO (onto or past GO). Then the

MOVE line is logged, followed by SALARY.

- **Backward moves** - a negative step count (card "Go back 3 squares") follows `prev` and never pays a salary.
- **MOVE_TO and NEAREST cards** - `findByName` or `findNearest` locates the target node; the recursive `forwardDistance` counts the steps; `step` then moves forward, so passing GO pays the salary (card K9 goes a full lap).
- **Go to Jail** - the player's position is set straight to `jailNode`: no walk along the ring, so no salary.
- **Rent for a full colour group** - the recursive `ownsGroup` walks once round from GO and stops at the first square of that colour the owner does not hold.
- **Board view** - BoardView walks `next` from GO to place the squares on screen; display only, never used for movement.

Code excerpt - Board.step (required recursive method)

code\Board.java

```
/**
 * Moves steps squares: along next if positive, along prev if negative.
 * Pays the salary (and flags it on p) when moving forward onto or past GO.
 * Base case: steps == 0 returns current.
 * Recursive case: one step towards zero, then recurse with steps - 1 or steps + 1.
 * Terminates because |steps| falls by 1 each call.
 */
public Node<Square> step(Node<Square> current, int steps, Player p) {
    if (steps == 0) {
        return current;
    }
    if (steps > 0) {
        Node<Square> next = current.next;
        if (next == goNode && p != null) {
            p.addCash(SALARY);
            p.setSalaryPaid(true);
        }
        return step(next, steps - 1, p);
    }
    return step(current.prev, steps + 1, p);
}
```

2.2 Linked queue - turn order and card decks

What it is. A first-in, first-out queue on singly linked nodes, with `front` and `rear` references. The same generic class is used three times: `LinkedQueue<Player>` for the turn order and two `LinkedQueue<Card>` for the decks.

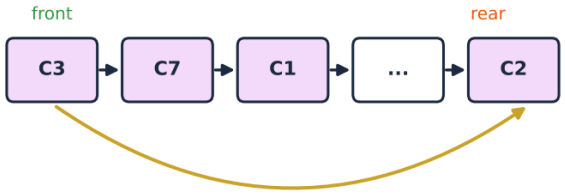
Why this structure. Both jobs are naturally FIFO: the player at the front plays and goes to the back; the top card is drawn and goes to the bottom. Both ends are O(1). An ArrayList was rejected because removing from the front shifts every element (O(n)); Java's ArrayDeque and LinkedList are banned by the brief.

Turn order - LinkedQueue<Player>



O(1) enqueue / dequeue / peek
remove(p): one recursive rotation,
other players keep their order - O(n)

Card decks - LinkedQueue<Card> (Chance and Community)



draw = dequeue(); apply effect; enqueue() at the back

Shuffled once at start: ArrayList + Collections.shuffle(list, seeded Random),
then enqueued in order.
JAIL_FREE is kept by the player (ArrayList<Card>) and
enqueued at the back of its own deck when used or on bankruptcy.
dequeue()/peek() on an empty queue throw IllegalStateException.

Game functions built on the queue

- **A turn** - `Game.playOneTurn`: `dequeue()` the front player, log TURN, apply the Jail rules or `takeTurn`, then `enqueue()` the player at the back unless bankrupt. The game ends when `turnQueue.size() <= 1` (LAST_PLAYER) or the turn limit is reached.
- **Removing a bankrupt player** - `remove(p)` rotates the queue exactly once with dequeue and enqueue (recursively) and drops p on the way, so the other players keep their order (brief 3.7 and 8.2).
- **PAY_EACH and COLLECT_EACH** - `copyInto` rotates the queue once to read the other players in turn-queue order (the current player is not in the queue during the turn); the payments then recurse over that list.
- **Decks** - cards are loaded into an ArrayList, shuffled once with the seeded Random (Chance first, then Community) and enqueued. Drawing is `dequeue()`, apply, `enqueue()` at the back. A JAIL_FREE card stays in the player's `jailFreeCards` list and returns to the back of its own deck when used or on bankruptcy.
- **Empty queue** - `dequeue()` and `peek()` throw IllegalStateException with a clear message (brief 4.1).
- **Twist T1 Reverse (if assigned)** - `reverse(q)` fits the class directly: dequeue one player, reverse the rest recursively, enqueue that player.

Code excerpt - LinkedQueue.remove (order-preserving removal)

code\LinkedQueue.java

```
/**
 * Removes the first occurrence of item (by reference) and keeps the order of
 * all other elements. Rotates the queue exactly once using only dequeue and
 * enqueue. Recursive: base case remaining == 0. O(n).
 */
public boolean remove(T item) {
    return removeRotate(item, size, false);
}

private boolean removeRotate(T item, int remaining, boolean removed) {
    if (remaining == 0) {
        return removed;
    }
    T x = dequeue();
    if (!removed && x == item) {
        removed = true;
    } else {
        enqueue(x);
    }
    return removeRotate(item, remaining - 1, removed);
}
```

2.3 ArrayList - indexed data with a changing size

The brief reserves `java.util.ArrayList` for data that needs indexed access and a changing size. Recursion over an ArrayList uses an index parameter, which is why `netWorth(owned, index)` takes one.

ArrayList	Where	Game function	Why ArrayList
ArrayList<Square> owned	Player	Buying adds a square; transit rent counts the owner's TRANSIT squares (recursive countOwned); net worth sums prices (recursive netWorth); bankruptcy returns every square (recursive, removing the last element each call, O(1))	Indexed recursion; grows as squares are bought
ArrayList<Player> players	Game	Join order; final ranking with Collections.sort (Player implements Comparable: higher net worth first, ties by join order); board view	Sorting and index access; java.util.Comparator is not on the allowed list
ArrayList<Card> (card list before shuffling)	FileLoader, Game	Cards in file order, split per deck, Collections.shuffle(list, seeded Random), then enqueued	Collections works on ArrayLists (brief 4.2)
ArrayList<Card> jailFreeCards	Player	Held Get Out of Jail Free cards; used first-in at the next jailed turn	Small list that changes size
ArrayList<Player> bankruptOrder	Game	Ranking lists bankrupt players most recent first	Read backwards by index
ArrayList<String> lines	GameLogger	Every Appendix C line, for the tests and the board view's event list	Append and read by index
ArrayList<Integer> scripted	Dice	Scripted dice values (--dice) for tests and demo scenarios	Read in order by index

2.4 Recursion - the eight required methods (brief 5.1)

None of these methods contains a for, while or do loop, and every helper they call is recursive or O(1). This is checked automatically by tests\check_constraints.py.

Method (class)	Game function	Base case	Recursive case	Why it terminates
step (Board)	Move by dice or card; pay salary at GO	steps == 0: return current	One hop along next (or prev), then step(.., steps - 1) (or + 1)	|steps| falls by 1 each call
takeTurn (Game)	One roll, again after a double	Third double: Jail, no move. Jailed or bankrupt after landing. Not a double	A double: takeTurn(p, doublesCount + 1)	doublesCount rises each call; the third double always stops
landOn (Game)	Resolve the square; follow card moves	Square did not move the player by a card, or player jailed/bankrupt	A card moved the player: landOn(p)	No card move ends on a card square (file checked): depth at most 2
transitRent (TransitSquare)	Rent 25 x 2^(n-1)	owned <= 1: 25	2 x transitRent(owned - 1)	owned falls by 1 to 1
netWorth (Player)	Property value for ranking	index >= size: 0	price at index + netWorth(owned, index + 1)	index rises to the list size
ownsGroup (Board)	Double rent for a full colour group	A property of the colour not owned by p: false. Back at the start: true	Check current, recurse on current.next	The ring is finite; the walk stops at start
findByName (Board)	MOVE_TO cards; card-file validation	remaining == 0: null. Name matches: node	Recurse on next, remaining - 1	remaining falls by 1
findNearest (Board)	NEAREST cards; finding Jail at set-up	remaining == 0: null. Type matches: node	Recurse on next, remaining - 1	remaining falls by 1

Recursive helpers

Helper	Purpose
LinkedQueue.removeRotate, copyRotate	Remove a player / read the queue in order with one rotation, order unchanged
Board.countForward (via forwardDistance)	Steps forward to a MOVE_TO or NEAREST target
Player.countOwned	Number of TRANSIT squares owned, for transit rent
Game.payEach, collectEach	PAY_EACH / COLLECT_EACH in turn-queue order; stop or bankrupt when someone cannot pay
Game.releaseSquares, returnJailFreeCards, clearCards	Bankruptcy: squares back to the bank, JAIL_FREE cards back to their decks
ConsoleBuyDecider.ask	Re-ask Y/N on invalid input without a loop inside landOn

Code excerpt - Game.landOn (required recursive method)

code\Game.java

```
/**
 * Required recursive method. Resolves the square the player stands on and
 * calls itself when a card moved the player to another square.
 * Base case: the square did not move the player by a card, or the player
 * is now in Jail or bankrupt.
 * Recursive case: a card moved the player -> landOn(p) on the new square.
 * Terminates because each recursive call needs a card move, and the card
 * file is checked so that no card move ends on a CHANCE or COMMUNITY
 * square: depth is at most 2.
 */
public void landOn(Player p) {
    p.setMovedByCard(false);
    p.getPosition().data.land(this, p);
    if (p.isMovedByCard() && !p.isBankrupt() && !p.isInJail()) {
        landOn(p);
    }
}
```

2.5 Supporting Java constructs

Construct	Where	Role
Abstract class and inheritance	Square > OwnableSquare > PropertySquare, TransitSquare; TaxSquare, CardSquare, CornerSquare	Each square type decides what happens on landing through land(Game, Player); landOn calls it polymorphically
Enumerations	DeckType, CornerType, CardEffect	Fixed sets of values; CardEffect drives a switch in Game.applyCard
Interface	BuyDecider: ConsoleBuyDecider (Y/N), AutoBuyDecider (--auto)	"The player decides whether to buy" (brief 3.4), interactive or scripted
java.util.Random (seeded)	Dice; Collections.shuffle	Same seed gives the same dice and card order, so the same game (brief 3)
java.util.Scanner	Main, ConsoleBuyDecider	Keyboard input
java.io	FileLoader (BufferedReader), GameLogger (PrintWriter)	Board and card files in; game log out
Plain Java arrays	BoardView screen buffer (char[][], String[][])	Drawing the screen only; not a collection and never holds game state

2.6 Compliance with the constraints (brief 4.2)

Rule	How the code meets it	Evidence
Own CircularDoublyLinkedList<T> and LinkedQueue<T>	Both written by the group with Node / QNode; no Java collection inside	Source; UML class diagram
A position is a Node<Square>; no index arithmetic or % size for movement	Player.position is a node; only Game.movePlayer and Game.sendToJail change it	Constraint check: no "% size"
From java.util only ArrayList, Scanner, Random, Collections	Imports limited to these four (plus java.io)	Constraint check: imports
No ArrayDeque, Stack, Vector, PriorityQueue, Deque, HashMap, TreeMap, HashSet, other collections, streams	None used; Comparable (java.lang) replaces java.util.Comparator	Constraint check: banned names
Required methods recursive, no loops; helpers recursive or O(1)	8 methods + 10 recursive helpers	Constraint check; design note D18
dequeue()/peek() on an empty queue throw a clear exception	IllegalStateException("... cannot dequeue from an empty queue")	Tests 1-2

2.7 Data flow through one turn (brief 8.1)

Step	Class.method	What happens	Data structure
Set-up	Main, FileLoader	Board file to Square objects; cards to a list; players added in join order	CDLL (addLast); ArrayList; shuffle; LinkedQueue (enqueue)
1	Game.playOneTurn	Front player taken; TURN logged	LinkedQueue.dequeue
2	Game.jailTurn (if jailed)	Use JAIL_FREE, or roll: double releases, third failure pays 50	ArrayList (held card); deck LinkedQueue.enqueue
3	Game.takeTurn	Roll two dice; ROLL logged; third double sends to Jail with no move	Recursion (doublesCount)
4	Game.movePlayer, Board.step	Walk the ring; salary at GO; MOVE then SALARY logged	CDLL next / prev; recursion
5	Game.landOn, Square.land	Buy, rent, tax, card or Go to Jail, by square type	Polymorphism; recursion for card moves
5a	landOnOwnable	Buy (owned.add) or pay rent (ownsGroup / transitRent)	ArrayList; CDLL walk; recursion
5b	drawCard, applyCard	Card drawn and returned; may move, pay, collect or jail	Deck LinkedQueue; recursion over players
6	declareBankrupt (if needed)	Cash to creditor; squares to bank; cards to decks; removed from queue	ArrayList (recursive); LinkedQueue.remove
7	Game.takeTurn	Double: roll again	Recursion
8	Game.playOneTurn	Player enqueued at the back unless bankrupt; end-of-game check	LinkedQueue.enqueue, size
End	Game.finish, rank	END and RANK lines; standings with properties	ArrayList sort; recursion (netWorth); CDLL walk

2.8 Game functionality and the data structure behind it

A one-line answer for every function built, as asked in brief Section 8.1 ("the role of each data structure ... and the cost of its operations").

Game functionality built	Data structure	Algorithm / method	Cost
Load the 24-square board	Circular doubly linked list	addLast per line; checks for one GO, JAIL, GO_TO_JAIL	O(n)
Load and shuffle the cards	ArrayList then LinkedQueue	Collections.shuffle (seeded), then enqueue	O(n)
Add players; turn order	ArrayList + LinkedQueue	add; enqueue in join order	O(1) each
Start a turn	LinkedQueue	dequeue the front player	O(1)
Move by the dice	Circular doubly linked list	recursive step along next	O(k)
Salary for passing or landing on GO	Circular doubly linked list	next == goNode inside step	O(1) per step
Move backward (card)	Circular doubly linked list	step with prev, no salary	O(k)
MOVE_TO / NEAREST cards	Circular doubly linked list	recursive findByName / findNearest + forwardDistance	O(n)
Go to Jail (square, card, third double)	Circular doubly linked list	direct jump to jailNode	O(1)
Doubles; third double	Recursion	takeTurn(p, doublesCount + 1)	at most 3 calls
Resolve a landing; card chains	Recursion + polymorphism	landOn then Square.land	depth at most 2
Buy a square	ArrayList	owned.add; owner set on the square	O(1)
Rent with colour-group doubling	Circular doubly linked list	recursive ownsGroup once round	O(n)
Transit rent	ArrayList + recursion	countOwned then transitRent	O(m)
Draw and return a card	LinkedQueue	dequeue, apply, enqueue	O(1)
Get Out of Jail Free	ArrayList + LinkedQueue	kept in player list; enqueued to its deck when used	O(1)
Pay each / collect each	LinkedQueue + ArrayList + recursion	copyInto rotation; recursive payEach / collectEach	O(p)
Jail attempts and fine	Player fields	attempt counter; third failure pays 50	O(1)
Bankruptcy	ArrayList + LinkedQueue	recursive release of squares and cards; remove by rotation	O(n)
End of game	LinkedQueue	size() <= 1 or turn limit	O(1)
Net worth and final ranking	ArrayList + recursion	netWorth recursion; Collections.sort with Comparable	O(p log p)
Game log (Appendix C)	ArrayList + file	GameLogger lines; PrintWriter	O(1) per line

Game functionality built	Data structure	Algorithm / method	Cost
Colour board view	Circular doubly linked list (read only)	walk next from GO into a screen grid	O(n)
Standings with properties	Circular doubly linked list + ArrayList	walk the ring filtered by owner; rank()	O(n x p)
Same game for the same seed	Random (seeded); ArrayList of scripted dice	one Random for dice and shuffle	-

n = squares on the board (24), k = squares moved, p = players (2-6), m = squares a player owns.

2.9 Prepared for the twists

The twists are deferred until the group's assignment is confirmed; a teammate's AI log names T1 Reverse, T5 Mirage and T8 Portals. Each one maps onto a structure that already exists:

- **T1 Reverse** - a new card effect; recursive `reverse(LinkedQueue)` using only enqueue, dequeue and isEmpty.
- **T5 Mirage** - a new Square subclass; moves back along `prev` with `step(-n)` (no salary) and re-enters `landOn`, with a per-turn counter capping Mirage moves at three.
- **T8 Portals** - a new Square subclass; recursive counts forward along `next` and backward along `prev` to the other portal, the shorter way taken (a tie goes forward); salary only when moving forward past GO.